DESIGN AND PROTOTYPING OF POST-QUANTUM AUTHENTICATION FOR SOME/IP SERVICE DISCOVERY

Omar MAHMOUD

Abstract

In this report, we cover the second phase of the research regarding securing the SOME/IP protocol with Post-Quantum Cryptography. As the automotive industry moves towards Ethernet-based Service-Oriented Architecture in inter-vehicle communication, securing the network communication becomes critical. In this phase we simulate an internal vehicle's network on an ARM64 processor to prove that the attack exists, and evaluate the solution. The results show that the ML-DSA-44 algorithm chosen in the first phase is in fact viable computationally, executing in an average of 0.252 ms, leaving the constraint of "MTU collision" that this research aims to fix.

Key words: Automotive Security, SOME/IP, Post-Quantum Cryptography, ML-DSA-44, Spoofing Attack, Ethernet MTU, Service Discovery.

1. Introduction

Following the problem description and definition of the state of the art in the first phase of this research, we now move on to practical testing in the second phase.

As the automotive industry transitions to Ethernet to support high-bandwidth applications like Advanced Driver Assistance Systems, video feeds, and other advanced driving technologies, it also required a transition to a new transport protocol, which is the Scalable service-Oriented Middleware over IP (SOME/IP) protocol. SOME/IP allows Electronic Control Units (ECUs) to act as clients and servers and dynamically find and subscribe to services.

The AUTOSAR, a global development partnership that establishes a standardized software architecture for automotive ECUs, does not define cryptographic authentication for the service discovery phase in the protocol, with the assumption that the internal network is a closed, trusted environment, however; if an attacker gains physical or remote access to the network (e.g., via a compromised infotainment system), then the network automatically trusts the attacker's messages as an internal component.

The primary objectives for this phase are as follows:

1. To demonstrate the real-world mechanics of SOME/IP routing hijack by exploiting the unauthenticated Service Discovery phase.
2. To benchmark the initial post-quantum candidate algorithm (ML-DSA-44) to prove it computes fast enough to meet strict automotive boot-time constraints, establishing a baseline for evaluating physical network viability.

2. Limitations of Current Authentication Standards

AUTOSAR does in fact define security mechanisms to protect SOME/IP communication, however, the current standards present limitations regarding internal threat models and strict automotive timing constraints

2.1 MACsec (Media Access Control Security)

MACsec operates at Layer 2 (Data Link) and is highly effective at encrypting the physical 100BASE-T1 copper Ethernet cables against external tapping, however, MACsec relies on a perimeter-based "zone of trust" model. It authenticates the hardware connection of an ECU, not the application-layer SOME/IP logic, which means that if an attacker can get access to a single ECU in the trusted network, MACsec will authenticate it and deliver malicious data, which is why application-layer authentication is still required[2].

2.2 DTLS (Datagram Transport Layer Security)

DTLS operates at Layer 4 (Transport) and secures the UDP packets carrying the SOME/IP payload, and while it’s very secure, DTLS requires a complex cryptographic handshake (exchanging certificates and negotiating session keys) before the first frame of data can be transmitted, a classical ECDSA DTLS handshake takes an average of 3.33 ms when tested on a powerful device, however, physical automotive sensor ECUs typically operate on highly constrained microcontrollers (e.g., 100 MHz - 300 MHz). When the baseline 3.33 ms cryptographic overhead is scaled to accurately reflect embedded hardware limitations, alongside the physical propagation delays of automotive switches, the handshake then violates the strict 50 ms "Time-to-First-Frame" (TTFF) safety regulation required for critical visual systems like backup cameras, proving that a low latency post quantum solution is required[3].

3. Simulation Environment and Hardware Setup

To ensure that the output of this phase is applicable to the automotive industry, the environment used for testing was designed to mirror actual ECU hardware constraints and standard network topologies.

3.1 Hardware Architecture

All of the simulation was executed on ARM64 architecture, modern vehicle ECUs are built on ARM architectures, ensuring the benchmarking of the cryptographic algorithms would provide an accurate reflection expected in a real vehicle.

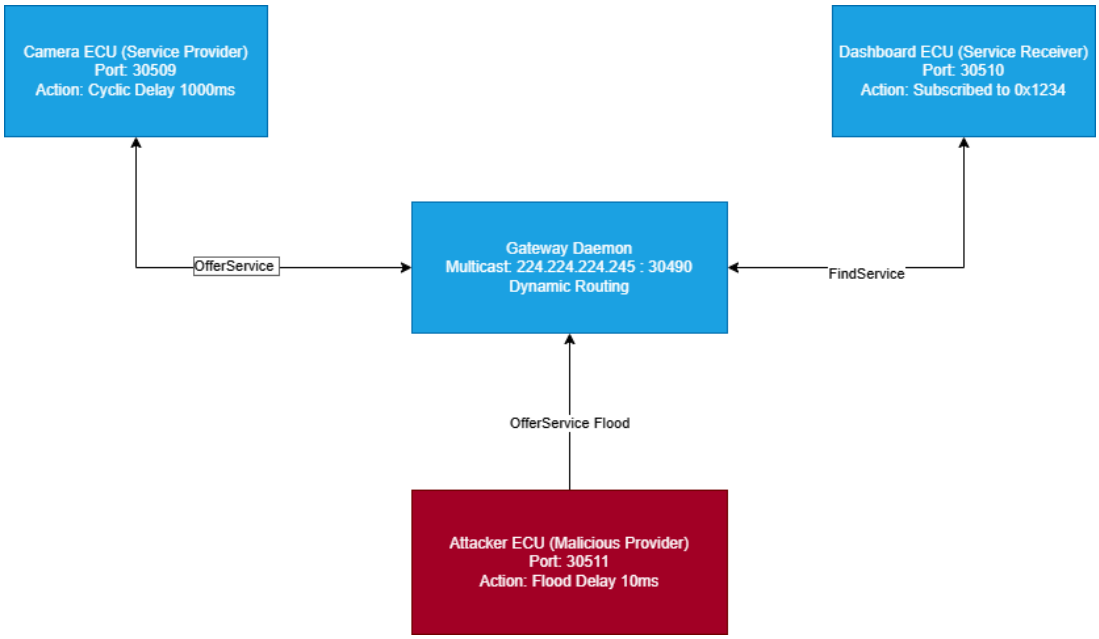
3.2 Network Topology and The Routing Daemon

The network stack was simulated within an Ubuntu Linux environment utilizing the someipy Python protocol library. In a physical vehicle utilizing the AUTOSAR Adaptive platform, ECUs do not manage raw socket connections directly. Instead, they communicate through a central middleware routing daemon [4].

The network was simulated using the `someipy` python protocol library in a linux environment. In a vehicle using the AUTOSAR Adaptive platform, ECUs do not manage the raw socket connections directly, they communicate through a central middleware routing daemon [4].

The simulation uses the someipyd daemon, and the Camera, Dashboard, and Attacker ECUs, communicate with this daemon using Unix Domain Sockets (UDS). The daemon is responsible for managing the actual UDP multicast groups (specifically 224.224.224.245 on port 30490 for Service Discovery), and standardized identifiers were used for the testing environment:

- **Service ID:** 0x1234 (Represents the Camera Feed being offered)
- **Instance ID:** 0x0001
- **EventGroup ID:** 0x4000 (The Video Stream subscription)
- **Event ID:** 0x8000 (The Payload)



4. Baseline Latency and The Attack Simulation

Before adding security extensions to SOME/IP, a baseline must be established for how fast the normal, unprotected network connects. Added security is useless if it delays the vehicle's normal functions.

4.1 Baseline Latency (Time to First Frame)

The first measurement is the "Time to First Frame" (TTFF), which measure the total time from the moment the dashboard ECU powers on, broadcasts a FindService request, receives the OfferService response from the camera, subscribes to the event group, and successfully receives the first payload frame.

The automotive industry's safety regulations dictate that critical visual systems, like the camera in this case, must boot and connect in 50 milliseconds or less [5], during the baseline testing, the unauthenticated network consistently completed the handshake and delivered the first frame in under 15 milliseconds, which leaves a 35 millisecond headroom for the addition of authentication to the protocol.

4.2 The Race Condition Exploit (Routing Hijack)

Because SOME/IP does not require ECUs to provide signatures during the Service Discovery phase, it is highly vulnerable to spoofing attack, the gateway daemon uses a "Last-In-Wins" rule to manage dynamic connections [1][6], while this is originally by design for fault tolerance so that backups can quickly replace main ECUs in case of errors or damage, an attacker can abuse this to hijack the service by broadcasting using the same Service ID.

We created an Attacker ECU that offers the same camera service as the real Camera ECU, which broadcasted its OfferService packet once every 1000 milliseconds (Cyclic offer delay), in order to hijack the connection, the attacker ECU was programmed to abuse this delay and flood the network with its packets.

```
# Accept the attack speed from the master script
delay_ms = int(sys.argv[1]) if len(sys.argv) > 1 else 100

# Configure the rogue service to exploit the Last-In-Wins logic
attacker_service = ServerServiceInstance(
    daemon=someipy_daemon, service=service, instance_id=0x0001,
    endpoint_ip="127.0.0.1", endpoint_port=30511, ttl=5,
    cyclic_offer_delay_ms=delay_ms # Dynamic speed
)
```

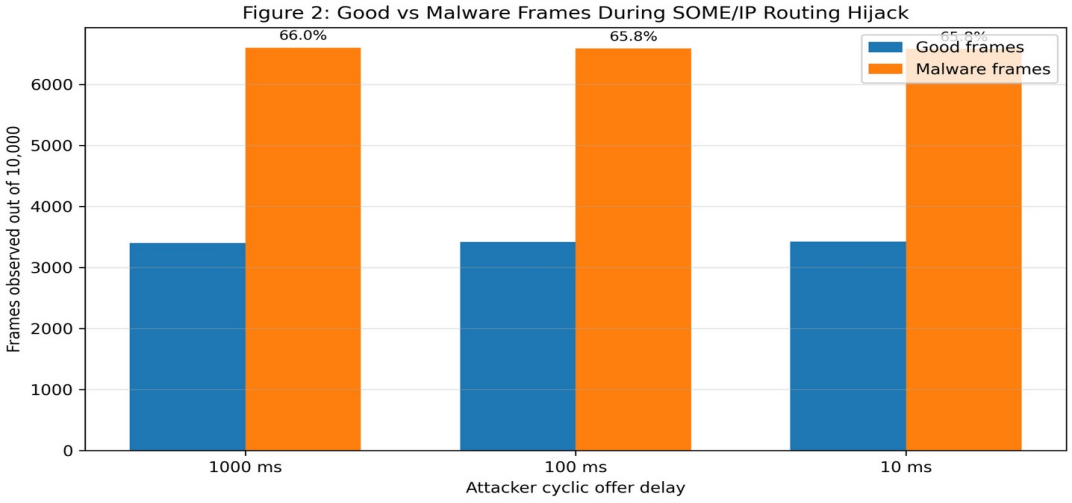
```
await attacker_service.start_offer()

try:
    while True:
        await asyncio.sleep(0.005)
        attacker_service.send_event(0x4000, 0x8000, b'HACKED_FRAME_MALWARE')
except asyncio.CancelledError:
    pass
```

Code Block 1: The high-frequency while loop utilized by the attacker ECU to flood the gateway and exploit the Last-In-Wins logic.

The attacker's delay is set between 10ms and 100ms, making it up to 100 times faster than the legitimate ECU. The gateway daemon constantly updated its internal routing table to point to the attacker's network port, making the dashboard unable to get the real ECU's feed.

Result: As shown in Figure 2, the high-frequency attacker successfully hijacked the dashboard connection, in over a test sample of 1,000 frames, the dashboard was forced to display malicious video frames about 66% of the time, which proves that an unauthenticated SOME/IP network cannot defend itself against a local attacker exploiting the Service Discovery repetition phase.



5. Testing the Post-Quantum Solution (ML-DSA-44)

In order to authenticate SOME/IP, digital signatures must be attached to the payload, and the dashboard must be able to verify that the payload came from a legitimate ECU before it displayed the feed, furthermore; since the lifecycle of

vehicles last for up to 15 and 20 years, the solution must also protected against future threats, like quantum computers able to break traditional cryptography.

5.1 Algorithm Selection: Why ML-DSA and not Falcon?

We selected the ML-DSA-44 algorithm (formerly Dilithium-2), which is the primary standard recommended by the National Institute of Standards and Technology (NIST) for general-purpose post-quantum digital signatures [7].

It is also important to justify why we did not choose FN-DSA (Falcon), another standard algorithm, despite the fact that it produces small signatures (666 bytes) that would easily fit inside ethernet packets and not have this collision problem. While libraries like PQM4 prove that Falcon can be successfully optimized to run on embedded automotive ARM microcontrollers[8], it relies on highly complex double-precision floating-point mathematics (Gaussian sampling over NTRU lattices) which is notoriously difficult to execute in "constant time"[9].

Furthermore, floating-point math is notoriously difficult to execute in "constant time." This makes the ECU highly vulnerable to timing side-channel attacks, where an attacker measures slight fluctuations in processor execution time to reverse-engineer the secret key [10], while ML-DSA, relies on simple, modular integer arithmetic. This makes it able to execute quickly on ARM processors and much easier to implement securely against side-channel attacks.

5.2 Speed Benchmark Methodology

In order to simulate the exact computational overhead of ML-DSA-44 that would be added to the network, we simulated the camera ECU singing a 32-byte dummy video frame, and the dashboard ECU utilizing the public key to verify the signature.

```
public_key, secret_key = generate_keypair()

message = b"SOME/IP_VIDEO_FRAME_PAYLOAD_DATA"

iterations = 1000

# Benchmark the ECU Signing Time

start_sign = time.time()

for _ in range(iterations):

    signature = sign(secret_key, message)

end_sign = time.time()

avg_sign_ms = ((end_sign - start_sign) / iterations) * 1000
```

```
        # Benchmark the Dashboard Verification Time
        start_verify = time.time()
        for _ in range(iterations):
            valid = verify(public_key, signature, message)
        end_verify = time.time()
        avg_verify_ms = ((end_verify - start_verify) / iterations) * 1000
```

Code Block 2: The stopwatch methodology used to benchmark the ML-DSA-44 algorithm over 1,000 continuous iterations.

To ensure the result is accurate, the program was run 1000 times and the average overhead was calculated.

5.3 Benchmark Results

Results:

Table 1: Empirical ML-DSA-44 Benchmark Results on ARM64

Metric	Result
Algorithm	ML-DSA-44
Average total overhead	0.252 ms
Signature size	2,420 bytes
Ethernet MTU reference	1,500 bytes

The initial results demonstrate that computationally, lattice-based cryptography is highly viable on ARM architecture, the ML-DSA-44 signing and verification process required only 0.252 ms, which when added to the 15 ms baseline network latency, the total connection time of 15.25 ms is still well below the 50 ms automotive safety requirement, this gives us a successful computational baseline, proving that quantum-resistant mathematics will not violate automotive boot-time constraints.

6. The MTU Collision Problem

While the algorithm's speed make it a viable option for authenticating SOME/IP, there is still the main issue that is the topic of this research. The ML-DSA-44 signature is 2420 bytes long, while the standard automotive Ethernet cable enforces a Maximum Transmission Unit (MTU) of 1500 bytes [11], if the signature is added to the SOME/IP packet, it will exceed the MTU and the package will be dropped.

Standard IP fragmentation is not a viable option in this case since SOME/IP utilizes UDP multicast to stream data, and standard IP fragmentation of UDP packets is highly unreliable in safety-critical systems, if a single fragment is dropped by a busy network switch, then the entire packet becomes corrupted, and because the application layer has no mechanism to recover the lost data, it is evident that ML-DSA-44 is not compatible with standard automotive Ethernet without severe modifications.

7. Comparison of Viable Post-Quantum Algorithms

Because ML-DSA-44 breaks the 1,500-byte MTU limit, an evaluation of alternative NIST post-quantum candidates is required to identify an algorithm that natively fits the automotive network architecture, a test of different algorithms was conducted in to benchmark signature generation alongside official NIST byte-size specifications.

Table 2: Viability of NIST Post-Quantum Algorithms for Automotive Ethernet

Algorithm	Category	Signature Size	Public Key Size	MTU Viability (1,500B)
ECDSA (secp256r1)	Classical Baseline	72 bytes	91 bytes	Native
ML-DSA-44	Lattice (Primary)	2,420 bytes	1,312 bytes	Fails (Collision)
Falcon-512	Lattice (Secondary)	655 bytes	897 bytes	Fits
MAYO-1	Multivariate	321 bytes	1,168 bytes	Fits
UOV (Level 1)	Multivariate	~96 bytes	~43,000 bytes	Fits (PK too large)
SLH-DSA-128	Hash-Based	17,088 bytes	32 bytes	Fails (Massive)

Analysis of Alternatives:

Hash-based signatures like SLH-DSA produce massive 17KB signatures, immediately disqualifying them for dynamic SOME/IP SD while Unbalanced Oil and Vinegar (UOV) produces an exceptionally small 96-byte signature, but its 43 KB public key presents a severe storage limitation for ECU controllers due to their small storage, MAYO presents a highly optimized balance (321-byte signature and 1.1 KB key), however, it has not been standardized yet.

FN-DSA (Falcon-512) is the most realistic architectural choice as it's a finalized NIST standard, the 655 bytes keysize does not face the MTU collision and while the empirical benchmark showed Falcon requires more computational time to sign a frame (2.43 ms) compared to ML-DSA (0.25 ms) due to its complex floating-point mathematics, open-source libraries such as PQM4 have proven that Falcon can be successfully optimized for automotive ARM Cortex-M microcontrollers[8].

8. Limitations of the Simulation

It is important to acknowledge the limitations of this testing phase, the attack simulation and benchmark were both run locally, and while the SOME/IP packets, byte structures, and gateway logic were accurate and compliant with the AUTOSAR standard, this software-level simulation does not account for the physical propagation of delays of real copper wiring or the delays of hardware network switches.

9. Conclusion and Future Work

In the second phase of this research, we successfully demonstrated that the SOME/IP protocol is highly vulnerable to routing hijacks due to its lack of application-layer authentication for Service Discovery and we established that while the primary NIST post-quantum standard (ML-DSA-44) executes efficiently on ARM processors, its 2,420-byte signature definitively breaks the standard 100BASE-T1 automotive Ethernet MTU limit, which makes it not a realistic choice for authentication of the protocol

For this reason, we must switch from ML-DSA to FN-DSA (Falcon). Because Falcon produces a 655-byte signature that fits within the 1,500-byte network constraint, so there will be no need for fragmentation.

Next Semester (R3) - The primary objective will be to fully integrate Falcon into the SOME/IP middleware, which involves modifying the gateway daemon to natively handle cryptographic signing and verification during the Service Discovery phase without breaking AUTOSAR compliance.

Final Semester (R4) - Transition from a local software simulation to a physical hardware environment, Utilizing optimized libraries like PQM4, the secured SOME/IP stack will be deployed onto physical microcontrollers to benchmark the true memory footprint, CPU cycle overhead, and physical latency limits.

BIBLIOGRAPHY

[1] AUTOSAR, "Specification of Service Discovery," AUTOSAR Foundation, Release 19-11, 2019.
[2] R. Lasar, et al., "Evaluation of MACsec and DTLS for Automotive Ethernet Networks," IEEE Automotive Networking Conference, 2022.
[3] National Highway Traffic Safety Administration (NHTSA), "Federal Motor Vehicle Safety Standards; Rear Visibility," 49 CFR Part 571.111, 2014.
[4] J. Seyler, et al., "Time-Sensitive Networking in Automotive E/E Architectures," IEEE Ethernet & IP @ Automotive Technology Day, 2015.
[5] M. Iehira, et al., "Spoofing Attack against SOME/IP and its Countermeasures," Proceedings of the IEEE Vehicular Technology Conference, 2018.
[6] K. T. Nguyen, et al., "A comprehensive study on automotive cybersecurity," IEEE Communications Surveys & Tutorials, 2020.
[7] National Institute of Standards and Technology (NIST), "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.
[8] M. Kannwischer, et al., "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," Second PQC Standardization Conference, 2019.
[9] T. Prest, et al., "FALCON: Fast-Fourier Lattice-based Compact Signatures over NTRU," NIST Post-Quantum Cryptography Standardization Project, 2020.
[10] J. Howe, et al., "Standard lattice-based key encapsulation on embedded devices," ACM Transactions on Embedded Computing Systems, 2020.
[11] IEEE Standard for Ethernet, "IEEE Std 802.3bw-2015 (100BASE-T1)," 2015.